

On the Efficiency of Sequentially Aware Recommender Systems: Cotten4Rec

Shankar Veludandi, *Rensselaer Polytechnic Institute, Troy, NY, 12180, USA*

Uzma Mushtaque, *Rensselaer Polytechnic Institute, Troy, NY, 12180, USA*

Gulrukh Kurdistan, *University of Michigan, Ann Arbor, MI, 48109, USA*

Abstract—Sequential recommendation (SR) models predict a user's next interaction by modeling their historical behaviors. Transformer-based SR methods, notably BERT4Rec, effectively capture these patterns but incur significant computational overhead due to extensive intermediate computations associated with Softmax-based attention. We propose Cotten4Rec, a novel SR model utilizing linear-time cosine similarity attention, implemented through a single optimized compute unified device architecture (CUDA) kernel. By minimizing intermediate buffers and kernel-launch overhead, Cotten4Rec substantially reduces resource usage compared to BERT4Rec and the linear-attention baseline, LinRec, especially for datasets with moderate sequence lengths and vocabulary sizes. Evaluations across three benchmark datasets confirm that Cotten4Rec achieves considerable reductions in memory and runtime with minimal compromise in recommendation accuracy, demonstrating Cotten4Rec's viability as an efficient alternative for practical, large-scale sequential recommendation scenarios where computational resources are critical.

Sequential recommender systems (SRSs) aim to predict the next item(s) a user will interact with by analyzing patterns and dependencies in their historical interactions. This task, also known as next-item prediction (NIP), has extensive applications across domains including e-commerce, social media, and streaming platforms. Capturing dynamic semantic contexts within user-item interactions is crucial for accurately modeling evolving user preferences, a capability predominantly achieved with transformer-based models. Models such as BERT4Rec and SASRec effectively utilize self-attention mechanisms inspired by the transformer architecture developed for natural language processing tasks[1], [2], [3], [4]. Transformer-based SR methods such as BERT4Rec attribute their success in recommendation accuracy to the implementation of softmax-scaled dot-product attention.

The operation is formally defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad (1)$$

where Q , K , and V are the query, key, and value matrices and d_k is the key dimension. Equation (1) explicitly forms an $s \times s$ similarity matrix whose storage cost grows quadratically with sequence length s . Current SR research predominantly emphasizes improving recommendation accuracy while largely overlooking computational efficiency, resulting in models that are unsuitable for scalable deployment. Numerous recent works focus on enhancing modeling power or training objectives – for example, by incorporating knowledge graphs and memory networks[5], [6], contrastive sequence representation learning objectives[7], [8], [9], counterfactual data augmentation[10], or even reinforcement learning to optimize long-term user engagement in recommendations[11]. These approaches have advanced accuracy on benchmark datasets but generally do little to address the growing time and memory costs. Linear-time attention mechanisms, introduced in the natural language processing (NLP) and computer vision (CV) communities, have been proposed to address these efficiency concerns. Notable examples include methods that approximate the softmax attention kernel with explicit feature mappings or low-rank projections, or use randomized feature tech-

niques to achieve linear complexity[12], [13], [14], [15]. Although such approaches avoid quadratic scaling, they typically still incur considerable memory overhead.

To overcome these limitations, we introduce Cotten4Rec, a novel SR model which replaces Eq. (1) with linear-time cosine similarity attention, termed "Cot-tention". Cotten4Rec uniquely leverages a single optimized CUDA kernel to fuse computations, significantly reducing GPU memory demands and eliminating the need for intermediate buffers. Cosine similarity attention, previously explored in textual and visual representation tasks, is introduced here for the first time within the SR domain. Unlike traditional linear-attention baselines, Cotten4Rec preserves the core advantages of self-attention mechanisms without requiring large matrix approximations, making it scalable for standard embedding dimensions and realistic short-to-moderate sequence lengths common in industry scenarios. This paper comprehensively evaluates Cotten4Rec against state-of-the-art (SOTA) BERT4Rec and the linear-attention baseline LinRec, providing critical insights into how reducing kernel-launch overhead and intermediate buffer allocation impact real-world sequential recommendation efficiency and practical performance.

Related Work

Early sequential recommenders often relied on recurrent neural networks or Markov chain models to capture user behavior over time. For example, the GRU4Rec model applied gated recurrent units to session-based recommendation sequences[16] and heuristic embedding approaches such as TransRec modeled item-to-item transitions as translation vectors in latent space[17]. These approaches demonstrated the value of sequence information, but their ability to model complex long-range dependencies was limited. The advent of transformers revolutionized SRSs by enabling high-capacity self-attention over user histories, allowing the model to focus on the most relevant parts of the input sequence for each word, effectively capturing long-range dependencies and enhancing the model's capability to process complex sequential data[3]. Subsequent SR models such as SASRec, the first to introduce a unidirectional transformer for sequential recommendation[2], and BERT4Rec, which improved on SASRec by applying a bidirectional transformer with a Cloze objective, both achieved state-of-the-art accuracy but inherited the $\mathcal{O}(s^2d)$ time and $\mathcal{O}(s^2)$ memory complexity of Eq. (1), which is prohibitive for moderately long interaction histories.

To mitigate the efficiency issue, researchers in NLP and CV have explored a variety of efficient attention

architectures. Many approaches strive for linear time attention by avoiding the full $s \times s$ attention matrix. For instance, Katharopoulos *et al.*[12] showed that transformers can be made pseudo-random access memory (RAM) friendly. The Linformer project demonstrated that projecting the sequence into a lower-dimensional subspace can reduce complexity[13]. Performers introduced random feature mappings to approximate the softmax kernel, reducing memory usage for long sequences[14]. More recently, the Long-Short Transformer combined local windowed attention with a global low-frequency approximation to handle long sequences efficiently[15]. In the sequential recommendation domain, LinRec leveraged L^2 -normalized linear attention to attain linear time complexity in SRS[18]. Linear-time variants—Performer [14], LinRec [18], and others—reduce the arithmetic to $\mathcal{O}(sd)$ yet still allocate auxiliary $s \times d$ buffers, giving an actual memory footprint of $\mathcal{O}(sd + d^2)$ in practice, where d is the embedding dimension.

Another line of research explores alternative similarity measures in attention. Cosine similarity has been used in various domains as a normalized measure of vector affinity. Recent work introduced a cosine attention formulation that normalizes queries and keys row-wise before computing attention weights, eliminating the need for expensive softmax operations[19]. Time and memory efficiency have thus emerged as critical considerations for sequential recommenders when dealing with longer user histories[20]. In this work, we generalize cosine similarity attention to the SRS setting and implement it in a single, highly optimized GPU kernel.

Methodology

Problem Definition

We focus on the NIP task in SR. Given a sequence of a user's past interactions

$$S = \{s_1, s_2, \dots, s_T\},$$

the goal is to predict the next item(s) the user is likely to interact with at time $T + 1$. Following BERT4Rec [1], we randomly replace a subset $\mathcal{M}$ of positions with the token [MASK] and optimize the negative log-likelihood

$$\mathcal{L} = - \sum_{(u,t) \in \mathcal{M}} \log p_{\theta}(s_t^{(u)} = v^* \mid S_{u, \setminus t}), \quad (2)$$

where v^* is the ground-truth item and θ denotes model parameters. During training, we randomly mask some positions in the sequence and train the model to predict the masked items based on the surrounding

context. The special `[MASK]` token replaces a certain fraction of positions in each training sequence, and the model attempts to identify the actual item at those positions. We use the negative log-likelihood loss as in BERT4Rec to optimize the probability of the correct item at each masked position. By corrupting sequences in this way, the model learns to infer any item in the sequence from its neighbors, effectively learning general sequential patterns rather than just prefix-to-next-item mappings. This bidirectional training scheme enables the model to utilize both earlier and later items in the sequence as context for prediction, resulting in more accurate recommendations compared to strictly forward-only models.

Cotten4Rec Architecture

Cotten4Rec's overall architecture is similar to that of Transformer-based recommenders, such as BERT4Rec, comprising an embedding layer, multiple self-attention blocks, and position-wise feed-forward layers. Each user history is first mapped to an item-embedding matrix plus a learnable positional embedding that preserves interaction order; this mirrors BERT4Rec and ensures the model has explicit sequence-order information. The key innovation lies in the self-attention mechanism within each block. Cotten4Rec maintains the Transformer stack but replaces Eq. (1) with cosine attention. For query q_i and key k_j , we first ℓ_2 -normalize both vectors and compute

$$\alpha_{i,j} = \frac{q_i \cdot k_j}{\|q_i\| \|k_j\|}, \quad (3)$$

which is bounded in $[-1, 1]$ where a higher score indicates greater similarity. By using cosine similarity, we eliminate the need for softmax normalization, as the attention scores are already bounded and comparatively scaled. Intuitively, the model attends to items whose embedding directions align most closely with the query's embedding direction. We additionally introduce a learnable scaling factor on the values v_j in the attention computation, akin to a global gain $m > 0$ that re-calibrates the spectrum of attention scores to ensure that the magnitude of the output remains appropriately tuned.

$$\alpha_{i,j}^{(\text{scaled})} = \frac{m q_i \cdot k_j}{\|q_i\| \|k_j\|}. \quad (4)$$

Cosine attention removes the exponential softmax and, when combined with kernel fusion, eliminates both the $s \times s$ score matrix and the additional $s \times d$ auxiliaries required by prior linear methods. Aside from the attention mechanism, Cotten4Rec retains the multi-head attention structure and other Transformer features found in BERT4Rec. Queries, keys, and values

are projected via learned linear layers, and multiple attention heads enable the model to attend to different aspects of the sequence in parallel. The output of the attention heads is combined and passed through a feed-forward network, along with residual connections, as usual. By maintaining this high-level architecture, we ensure that any accuracy differences primarily stem from the attention mechanism rather than unrelated architectural changes.

Single-Kernel Efficiency

The most significant practical improvement in Cotten4Rec stems from the implementation of attention computation. Traditional transformers launch three successive kernels to evaluate Eq. (1): computing the $QK^\top$ similarity matrix, applying the softmax function, and then multiplying by V . Separate GPU kernels might execute each of these steps and can require storing large intermediate matrices, such as the full attention score matrix of size $s \times s$. This results in substantial overhead due to repeated kernel launches and the heavy use of global memory bandwidth to read and write intermediate results. While LinRec's exponential linear unit (ELU) kernel still allocates an extra $s \times d$ buffer and fires several smaller CUDA kernels, Cotten4Rec avoids these inefficiencies by performing the calculations implied by Eqs. (3)–(4) within a single CUDA kernel that streams (Q, K, V) , computes $\alpha_{i,j}^{(\text{scaled})}$ on the fly, and directly accumulates the weighted values in registers or shared memory.

This fusion drastically reduces overhead because the GPU does not have to pause and reschedule work for multiple small tasks; instead, it stores temporary results in fast on-chip memory (registers or shared memory) rather than writing them out to global memory. Consequently, Cotten4Rec requires far less memory per training step. Notably, we do not allocate an $s \times s$ buffer for attention scores at all, which cuts out the principal source of quadratic memory growth. We also avoid the extra buffers that linear attention methods use to allocate auxiliary matrices for intermediate computations. The time saved from fewer kernel launches is significant on modern GPUs, as each kernel launch incurs a fixed cost (on the order of tens of microseconds), which adds up when a model calls many small operations for every batch. By paying this cost only once per attention block, Cotten4Rec speeds up training, especially in cases where the sequence length s is short to moderate. In summary, this design removes the main $\mathcal{O}(s^2)$ storage term and reduces launch overhead, giving the practical memory profile of $\mathcal{O}(sd + d^2)$.

TABLE 1. Summary of datasets including user and item statistics, sequence length, and density.

Dataset	# Users	# Users After Preprocessing	# Items	# Interactions	Average Sequence Length	Density
ML-1M	6,040	6,040	3,706	1,000,209	166	95.53%
Beauty	1,210,271	52,361	120,472	466,364	9	99.99%
ML-20M	138,493	111,894	16,569	7,573,787	68	99.59%

TABLE 2. Comparison of Cotten4Rec performance across various sequence lengths with Bert4Rec and LinRec models

Dataset	Seq. Len.	Model	Memory	Time	NDCG	HIT	Δ Memory	Δ Time	Δ NDCG	Δ HIT
Beauty	20	BERT4Rec	5246	1359	0.249	0.395	-22.0%	-11.4%	-1.8%	-0.8%
		LinRec	5269	1210	0.248	0.394	-22.3%	-0.5%	-1.5%	-0.5%
		Cotten4Rec	4094	1204	0.244	0.392				
	50	BERT4Rec	12473	1285	0.250	0.396	-23.6%	0.3%	-2.2%	-1.0%
		LinRec	12499	1319	0.248	0.393	-23.8%	-2.4%	-1.3%	-0.2%
		Cotten4Rec	9528	1288	0.245	0.392				
	100	BERT4Rec	24521	1556	0.248	0.392	-24.2%	-2.8%	-1.0%	0.0%
		LinRec	24534	1583	0.248	0.393	-24.3%	-4.4%	-1.1%	-0.2%
		Cotten4Rec	18578	1513	0.246	0.392				
ML-1M	50	BERT4Rec	667	266	0.381	0.609	-19.0%	51.9%	-16.8%	-11.2%
		LinRec	692	297	0.357	0.588	-21.9%	36.1%	-11.1%	-8.0%
		Cotten4Rec	540	404	0.318	0.541				
	100	BERT4Rec	1318	273	0.371	0.602	-21.1%	56.4%	-15.4%	-10.0%
		LinRec	1332	296	0.346	0.580	-21.9%	44.3%	-9.1%	-6.6%
		Cotten4Rec	1040	427	0.314	0.542				
	200	BERT4Rec	2689	298	0.358	0.593	-24.1%	58.5%	-13.4%	-9.3%
		LinRec	2612	316	0.322	0.561	-21.9%	49.2%	-3.8%	-4.1%
		Cotten4Rec	2040	472	0.310	0.538				
ML-20M	50	BERT4Rec	2998	777	0.750	0.932	-22.8%	-9.4%	-1.8%	-0.6%
		LinRec	3023	818	0.749	0.932	-23.4%	-13.9%	-1.6%	-0.6%
		Cotten4Rec	2315	704	0.737	0.927				
	100	BERT4Rec	5895	1350	0.755	0.936	-23.6%	-22.2%	-2.1%	-0.6%
		LinRec	5909	1279	0.752	0.936	-23.8%	-17.9%	-1.8%	-0.6%
		Cotten4Rec	4501	1050	0.739	0.931				
	200	BERT4Rec	11769	2267	0.756	0.938	-24.6%	-24.0%	-2.4%	-0.5%
		LinRec	11694	2247	0.750	0.936	-24.1%	-23.4%	-1.6%	-0.4%
		Cotten4Rec	8877	1722	0.738	0.933				

Experimental Setup

Before evaluating Cotten4Rec, we ensure a fair comparison with baseline models. We use the exact embedding dimensions, sequence length cutoffs, and training strategy for Cotten4Rec, BERT4Rec [1], and LinRec [18]. This way, any differences in performance can be attributed to the attention mechanisms and implementation efficiencies rather than other factors. We tested all models on three public benchmark datasets: Amazon Beauty (Beauty), MovieLens-20M (ML-20M), and MovieLens-1M (ML-1M), which span short ($s \approx 9$), medium ($s \approx 68$), and long ($s \approx 166$) history cases at realistic catalog sizes as per Table 1. Table 1 summarizes key statistics for each dataset, including user and item counts, average sequence length, and dataset density. By evaluating each model across a range of sequence lengths and domains, we can observe how they scale and perform under different conditions.

Results & Findings

Table 2 reports four key metrics for each model: maximum graphics processing unit (GPU) memory (Mem) in megabytes, training time per epoch (Time) in seconds, and two ranking-quality scores—normalized discounted cumulative gain at rank 10 (NDCG@10) and hit rate at rank 10 (HIT@10). NDCG@10 rewards recommended lists that place relevant items near the top, while HIT@10 simply measures whether at least one relevant item appears in the top-10. For each dataset and sequence length we also show how much better or worse Cotten4Rec is relative to BERT4Rec and LinRec based on the key metrics; negative values for Δ Mem, Δ Time indicate Cotten4Rec is more efficient while negative values for Δ NDCG, Δ HIT indicate Cotten4Rec is less accurate. The average percent difference approximations later throughout this section are based off the percent differences in Table 2.

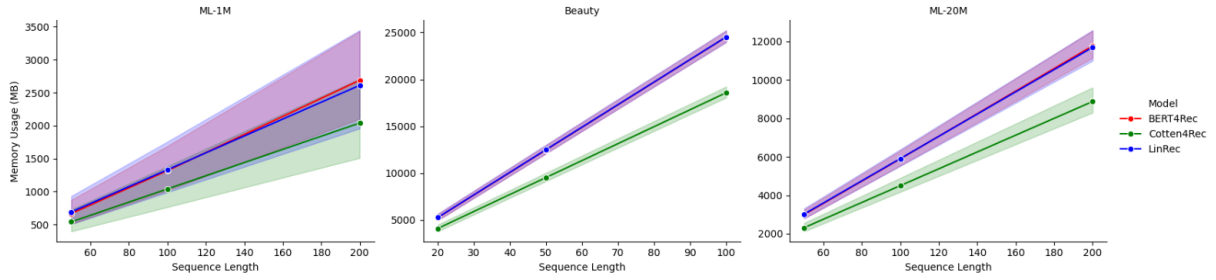


FIGURE 1. GPU Memory Usage vs. Sequence Length - Cotten4Rec uses significantly less memory than baselines as sequence length grows.

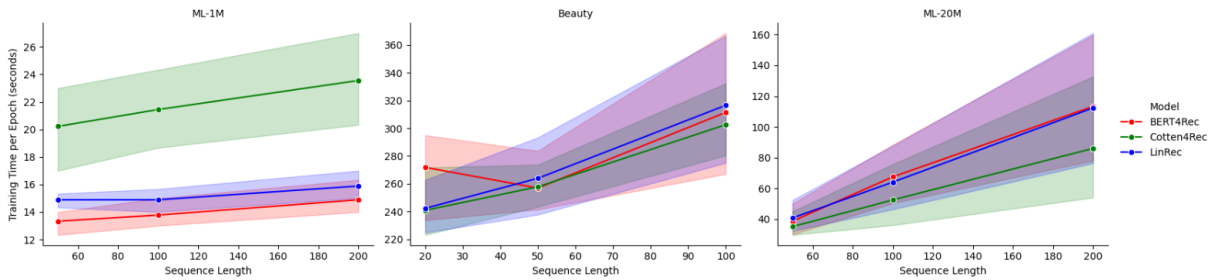


FIGURE 2. Training Time per Epoch vs Sequence Length - Cotten4Rec is slower than the baselines on ML-1M, but is faster on Beauty and ML-20M.

Memory Efficiency

Figure 1 displays how Cotten4Rec achieves substantial reductions in GPU memory usage, cutting peak memory requirements by approximately 23% compared to both BERT4Rec and LinRec across all evaluated datasets. This improvement aligns closely with its fused cosine-attention kernel design, which fundamentally avoids the quadratic memory overhead inherent in BERT4Rec’s softmax-based attention, where large $s \times s$ score matrices must be explicitly formed and stored. Unlike LinRec, which utilizes an ELU+1 linear kernel that still materializes an additional $s \times d$ feature map, Cotten4Rec integrates row-wise normalization of key and value matrices within a single CUDA kernel, accumulating results directly in GPU registers and shared memory. Consequently, Cotten4Rec significantly reduces both intermediate buffer allocation and global memory traffic, demonstrating practical advantages that surpass theoretical big-O complexity considerations for realistic sequence lengths.

Training Efficiency

Figure 2 shows how the training time impact of Cotten4Rec varies across datasets. On Beauty and ML-20M, it reduced training time by approximately 4% and

20%, respectively, relative to both baselines. These speedups are directly attributable to Cotten4Rec’s streamlined, single-kernel attention computation that eliminates repeated kernel launches and intermediate global memory accesses common in the multi-kernel approaches of BERT4Rec and LinRec. ML-1M, however, tells a different story: Cotten4Rec is approximately 49% slower than both baselines on average. The slowdown stems from a mismatch between ML-1M’s very long sequences ($s \approx 166$) and Cotten4Rec’s per-token cosine arithmetic. Once s becomes large, arithmetic cost—not memory traffic—dominates, so the fused kernel’s launch savings can no longer compensate. Note that ML-1M’s vocabulary is relatively small, so the embedding lookup—identical across models—no longer hides attention latency.

Recommendation Quality

In terms of recommendation accuracy, Cotten4Rec exhibited a marginal reduction in performance, trailing BERT4Rec and LinRec by at most 2% on Beauty and ML-20M and more significantly by 11% and 8% in NDCG@10 and HIT@10 on ML-1M, respectively. This reduction is primarily due to the different characteristics of the attention mechanisms: BERT4Rec’s and

LinRec's approaches inherently capture fine-grained dependencies using a softmax or linear approximation with richer interactions, while Cotten4Rec's cosine based attention, while computationally more efficient, slightly simplifies interaction modeling by normalizing attention scores. Nonetheless, the observed accuracy reduction is relatively modest for shorter to medium-length sequences, suggesting that Cotten4Rec achieves a favorable trade-off, providing substantial computational efficiency at the expense of only minimal accuracy loss. For practitioners, a 2 % hit in NDCG may be acceptable if it unlocks 20 % faster training or lets the model fit on commodity GPUs.

Conclusion

Cotten4Rec shows that careful kernel-level engineering can yield outsized returns for production-scale recommenders. When sequence length is modest, constant-factor savings from fewer kernel launches and in-register computation outweigh raw FLOP counts, but as s grows, arithmetic complexity begins to dominate.

Several practical limitations follow. First, Cotten4Rec's speed advantage is sequence-length sensitive; beyond roughly two hundred tokens, the fused cosine kernel's extra arithmetic eclipses its launch savings, reversing the runtime win. Second, the implementation targets NVIDIA GPUs and has not been ported to other accelerators, so its efficiency assumptions about register and shared-memory sizes may not hold universally. Third, while cosine attention preserves most accuracy on short-to-medium traces, it loses expressiveness on longer sequences, leaving a measurable gap against softmax baselines. These factors set an upper bound on the model's scalability and highlight the need for a more adaptive approach.

Two directions are the most promising for future work. A fully fused softmax kernel that performs exponentiation, normalization, and value aggregation in one pass could restore softmax's expressiveness without re-introducing large intermediate buffers, though designing such a numerically stable kernel that never spills the $s \times s$ score block remains a formidable challenge. Complementarily, an adaptive strategy that switches between softmax and cosine (or other linear) attention on a per-layer or per-sequence basis could deliver the best of both worlds—lean speed for typical traces and richer modeling capacity when user histories grow long. Pursuing these avenues will move sequential recommenders toward architectures that are simultaneously accurate, resource-frugal, and robust across the full spectrum of real-world sequence lengths.

Acknowledgements

The authors thank Advait Pavuluri for assistance with experimental runs.

REFERENCES

1. F. Sun *et al.*, "Bert4rec: Sequential recommendation with bidirectional encoder representations from transformers," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manag. (CIKM)*, 2019, pp. 1441–1450.
2. W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2018, pp. 197–206.
3. A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
4. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL)*, 2019, pp. 4171–4186.
5. J. Huang, W. Zhang, X. Zhang, and Y. Yu, "Improving sequential recommendation with knowledge-enhanced memory networks," in *Proc. 41st Int. ACM SIGIR Conf. Res. Dev. Inf. Retrieval*, 2018, pp. 505–514.
6. J. Tang *et al.*, "Towards neural mixture recommender for long range dependent user sequences," in *Proc. Web Conf. (WWW)*, 2019, pp. 1782–1793.
7. X. Xie *et al.*, "Contrastive learning for sequential recommendation," 2020, arXiv:2010.14395.
8. R. Qiu *et al.*, "Contrastive learning for representation degeneration problem in sequential recommendation," in *Proc. 15th ACM Int. Conf. Web Search Data Mining (WSDM)*, 2022, pp. 813–823.
9. Y. Chen *et al.*, "Intent contrastive learning for sequential recommendation," in *Proc. Web Conf. (WWW)*, 2022, pp. 2172–2182.
10. S. Zhang *et al.*, "Causerec: Counterfactual user sequence synthesis for sequential recommendation," in *Proc. 44th Int. ACM SIGIR Conf.*, 2021, pp. 367–377.
11. X. Zhao *et al.*, "Dear: Deep reinforcement learning for online advertising impression in recommender systems," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 1, 2021, pp. 750–758.
12. A. Katharopoulos *et al.*, "Transformers are rnns: Fast autoregressive transformers with linear attention," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, 2020.
13. S. Wang *et al.*, "Linformer: Self-attention with linear complexity," 2020, arXiv:2006.04768.
14. K. Choromanski *et al.*, "Rethinking attention with

- performers,” in *Int. Conf. Learn. Represent. (ICLR)*, 2021.
15. C. Zhu *et al.*, “Long-short transformer: Efficient transformers for language and vision,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.
 16. B. Hidasi *et al.*, “Session-based recommendations with recurrent neural networks,” in *Int. Conf. Learn. Represent. (ICLR)*, 2016.
 17. R. He *et al.*, “Translation-based recommendation,” in *Proc. 11th ACM Conf. Recommender Syst. (RecSys)*, 2017, pp. 161–169.
 18. L. Liu *et al.*, “Linrec: Linear attention mechanism for long-term sequential recommender systems,” in *Proc. 46th Int. ACM SIGIR Conf.*, 2023, pp. 289–299.
 19. G. Mongaras *et al.*, “Cottention: Linear transformers with cosine attention,” 2024, arXiv:2409.18747.
 20. S. Wang *et al.*, “Sequential recommender systems: Challenges, progress and prospects,” in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 6332–6338.

Shankar Veludandi is currently a Researcher in the Department of Computer Science at Rensselaer Polytechnic Institute, Troy, NY, USA. His research interests include efficient architectures for sequential recommender systems, attention mechanisms in deep learning, and practical optimization of large-scale machine learning models. Veludandi recieved an M.S. in computer science from Rensselaer Polytechnic Institute. He is a Student Member of IEEE and the IEEE Computer Society. Contact him at href-mailto:shankar.veludandi.02@gmail.com

Uzma Mushtaque is currently a Senior Lecturer in the Department of Computer Science at Rensselaer Polytechnic Institute, Troy, NY, USA. Her research focuses on efficient foundation models for sequential recommender systems, computing-education methods that prepare the future workforce, and time-series forecasting for financial data. Mushtaque earned a Ph.D. in decision sciences and engineering systems from Rensselaer Polytechnic Institute. She is a Member of the IEEE, the IEEE Computer Society, and the ACM. Contact her at mushtu@rpi.edu.

Gulrukh Kurdistan is currently a Ph.D. Candidate in computer science and engineering at the University of Michigan, Ann Arbor, USA. Her research interests include efficient transformer architectures for sequential recommendation, scalable large-language-model inference, and applied machine learning for consumer analytics. Kurdistan received an M.S. in computer science from Rensselaer Polytechnic Institute. She is a Student Member of the IEEE and a member of the ACM. Contact her at gulrukhk@umich.edu.